

Python Functions: Advanced

Comprehensive Reference & Practical Guide

A function is a block of code which only runs when it is called. You can pass data, known as parameters, into a function. A function can return data as a result.

1. Function Arguments

Information can be passed into functions as arguments. Arguments are specified after the function name, inside the parentheses.

Example: Positional vs Keyword

```
def describe_pet(animal_type, pet_name):  
    print(f"I have a {animal_type} named {pet_name}.")  
  
# Positional arguments  
describe_pet("Hamster", "Harry")  
  
# Keyword arguments  
describe_pet(pet_name="Winston", animal_type="Dog")
```

2. Arbitrary Arguments (*args)

If you do not know how many arguments will be passed into your function, add a * before the parameter name. The function will receive a **tuple** of arguments.

Example: calculate_sum(*numbers)

```
def calculate_sum(*numbers):
    total = 0
    for n in numbers:
        total += n
    return total

print(calculate_sum(10, 20, 30)) # Result: 60
```

3. Arbitrary Keyword Arguments (**kwargs)

If you do not know how many keyword arguments will be passed, add ****** before the parameter name. The function will receive a **dictionary** of arguments.

Example: Dynamic Profile Building

```
def build_profile(first, last, **user_info):
    user_info['first_name'] = first
    user_info['last_name'] = last
    return user_info

user = build_profile('Marie', 'Curie', location='Paris',
                    field='Radioactivity')
print(user)
```

4. Python Scope

A variable is only available from inside the region it is created. This is called **scope**.

- **Local Scope:** Created inside a function; accessible only there.
- **Global Scope:** Created in the main body; accessible everywhere.

The global Keyword

```
x = 300

def myfunc():
    global x
    x = 200
    print("Inside:", x)

myfunc()
print("Outside:", x)
```

5. Lambda Functions

A lambda function is a small anonymous function. It can take any number of arguments, but can only have one expression.

Example: Lambda in a Function

```
def my_multiplier(n):
    return lambda a : a * n

mydoubler = my_multiplier(2)
print(mydoubler(11)) # Output: 22
```

6. Decorators

Decorators modify the behavior of a function by "wrapping" it. This is useful for adding functionality like logging or timing without altering the original code.

Example: Timer Decorator

```
import time

def timer(func):
    def wrapper(*args, **kwargs):
        start = time.time()
        res = func(*args, **kwargs)
        print(f"Time: {time.time() - start}s")
        return res
    return wrapper

@timer
def wait():
    time.sleep(1)

wait()
```

7. Recursion

Recursion is when a function calls itself. It's often used for mathematical sequences or traversing complex structures.

Example: Factorial

```
def factorial(n):  
    if n == 1:  
        return 1  
    else:  
        return n * factorial(n-1)  
  
print(factorial(5)) # Result: 120
```

8. Generators

Generators use `yield` instead of `return`. They produce items one at a time and are very memory efficient.

Example: Simple Generator

```
def my_gen(n):  
    while n > 0:  
        yield n  
        n -= 1  
  
for val in my_gen(3):  
    print(val)
```

Practical Hands-on Exercise

Challenge: The Processor Pipeline

```
def logger(func):
    def wrapper(*args):
        print("Starting Pipeline...")
        return func(*args)
    return wrapper

@logger
def process_data(*args):
    clean = lambda x: str(x).strip().upper()
    for item in args:
        yield clean(item)

# Run pipeline
for entry in process_data(" python ", " java", 101):
    print(entry)
```